



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

CURSO DE PROCESAMIENTO DIGITAL DE SEÑALES

Desarrollo de una FFT de 1024 puntos mediante mariposas

Ortiz Díaz Miguel Ángel

Reza Monroy Gustavo

Zúñiga Dominguez Kevin Alexander

Profesor: Dr. Oscar Francisco Fuentes Casarrubias

7 de octubre de 2026

1. Introducción

El análisis de señales en el dominio de la frecuencia es un pilar indispensable en la ingeniería electrónica y de telecomunicaciones. Desde que Jean-Baptiste Joseph Fourier postuló en el siglo XIX que cualquier señal periódica o fenómeno físico continuo podía representarse mediante una sumatoria infinita de funciones senoidales y cosenoidales, la forma en que comprendemos las variaciones en el tiempo cambió para siempre. En la ingeniería moderna, esta perspectiva frecuencial nos permite identificar armónicos en líneas de transmisión eléctrica, aislar ruido en sistemas de instrumentación acústica, y demodular canales en espectros de radiofrecuencia saturados.

Cuando transitamos del análisis de señales continuas a la era de la computación digital, se hizo necesario adaptar estas matemáticas a sistemas discretos. La herramienta matemática estándar para esta conversión, cuando se trata de secuencias muestreadas finitas, es la Transformada Discreta de Fourier (DFT). Esta operación establece una correspondencia exacta entre las muestras discretas en el tiempo y un número equivalente de muestras espectrales en la frecuencia. Cada punto resultante en el espectro nos indica la amplitud y la fase de una frecuencia específica contenida dentro de la señal original muestreada.

A pesar de su elegancia teórica, la implementación práctica de la DFT presentó un cuello de botella abrumador durante las primeras décadas de la computación. El desafío histórico del procesamiento digital ha radicado en que la DFT requiere evaluar cada una de las componentes frecuenciales recorriendo iterativamente todas las muestras temporales. Esto significa que para encontrar el valor de un solo punto en el espectro de frecuencias, el procesador debe realizar multiplicaciones y sumas con cada uno de los datos capturados en el tiempo. Para secuencias de datos largas, esto resulta en una complejidad algorítmica ineficiente para los procesadores, limitando severamente las aplicaciones en tiempo real.

La magnitud de este problema se evidencia al analizar el crecimiento exponencial del tiempo de cómputo. Una señal de audio capturada durante apenas un segundo a una tasa de muestreo estándar de 44.1 kHz requeriría cerca de dos mil millones de operaciones matemáticas para ser transformada al dominio de la frecuencia mediante una DFT tradicional. En escenarios críticos como la decodificación de radares militares o la sismología, esperar horas por el análisis espectral era inaceptable.

Para resolver esta barrera, surge la Transformada Rápida de Fourier (FFT). Popularizada en 1965 por James Cooley y John Tukey, la FFT no es simplemente una mejora de hardware, sino una revolución algorítmica. Es fundamental aclarar que la FFT no es una transformada matemática distinta a la DFT, sino un algoritmo rigurosamente optimizado que explota la simetría y la periodicidad del núcleo de la transformada para reducir drásticamente esta complejidad computacional. En lugar de recalcular combinaciones redundantes de senos y cosenos, la FFT reutiliza resultados intermedios de manera inteligente.

El impacto de esta optimización es inconmensurable en el mundo contemporáneo. Sin la reducción de operaciones que provee la FFT, tecnologías fundamentales como las redes inalámbricas Wi-Fi y 5G (que utilizan multiplexación por división de frecuencias ortogonales, u OFDM), la compresión de imágenes JPEG, los formatos de audio MP3 y los escáneres de resonancia magnética (MRI) serían tecnológicamente inviables. El objetivo del presente documento es deconstruir la arquitectura interna

de este algoritmo, analizando paso a paso cómo una estructura de cálculo masivo se subdivide en etapas eficientes mediante operaciones elementales conocidas como mariposas, aplicadas a un caso práctico de 1024 puntos.

2. Planteamiento del problema

La Transformada Discreta de Fourier para una secuencia discreta de longitud $N = 1024$ puntos se define mediante la siguiente sumatoria analítica:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad k = 0, 1, \dots, N-1 \quad (1)$$

Al analizar los requerimientos del sistema, observamos que el número de puntos seleccionados, $N = 1024$, cumple con una condición crucial: es una potencia exacta de dos, ya que matemáticamente $1024 = 2^{10}$.

El reconocimiento de que $1024 = 2^{10}$ dicta la elección del esquema algorítmico a utilizar: el algoritmo Radix-2 (base 2). Este algoritmo emplea un enfoque de “divide y vencerás”, fraccionando la secuencia iterativamente a la mitad (pares e impares). Por lo tanto, la resolución completa de la FFT puede implementarse exactamente en 10 etapas computacionales escalonadas.

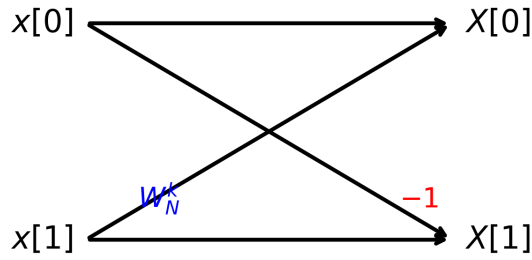
3. Desarrollo por mariposas

El corazón operativo del algoritmo FFT es una subrutina fundamental conocida como “mariposa” (butterfly). En lugar de procesar la sumatoria completa de manera lineal, la operación básica de mariposa agrupa matemáticamente muestras de dos en dos en cada etapa, sumándolas y restándolas sistemáticamente tras ser escaladas por los factores de giro (*twiddle factors*) correspondientes.

Los factores de giro, que representan vectores rotatorios complejos en el círculo unitario, se definen como:

$$W_N^k = e^{-j\frac{2\pi k}{N}} = \cos\left(\frac{2\pi k}{N}\right) - j \sin\left(\frac{2\pi k}{N}\right) \quad (2)$$

El flujo de datos entra a la mariposa de la siguiente manera: se toman dos puntos complejos de la salida de la etapa anterior; la rama inferior se multiplica por el factor de giro W_N^k , y finalmente se calculan dos nuevas salidas convergentes. La salida de la rama superior se genera mediante una suma, y la salida de la rama inferior se genera mediante una resta.



4. Estructura de las 10 etapas

Para mapear de manera exhaustiva una señal de entrada de 1024 puntos, la estructura algorítmica se organiza rigurosamente en 10 etapas de procesamiento secuencial. En cada una de estas etapas se deben calcular exactamente $\frac{N}{2} = 512$ operaciones de mariposa independientes.

La principal variable estructural es la forma en que cambian las distancias entre los índices de las muestras involucradas en las mariposas a medida que se avanza en cada nivel computacional. El espaciamiento continúa duplicándose invariablemente a lo largo del proceso algorítmico. La progresión exacta para las 10 etapas es la siguiente:

- **Etapas 1:** La separación entre los datos cruzados es de $2^0 = 1$ (muestras adyacentes).
- **Etapas 2:** La distancia aumenta a $2^1 = 2$.
- **Etapas 3:** La distancia se expande a $2^2 = 4$.
- **Etapas 4:** La distancia de cruce se duplica a $2^3 = 8$.
- **Etapas 5:** La distancia de cruce se duplica a $2^4 = 16$.
- **Etapas 6:** La distancia de cruce se duplica a $2^5 = 32$.
- **Etapas 7:** La distancia de cruce se duplica a $2^6 = 64$.
- **Etapas 8:** La distancia de cruce se duplica a $2^7 = 128$.
- **Etapas 9:** La distancia de cruce se duplica a $2^8 = 256$.
- **Etapas 10:** En el nivel final, la distancia alcanza su máximo de $2^9 = 512$.

Esta estructura piramidal cruzada permite que la información de cada una de las 1024 muestras originales contamine algorítmicamente a todas las 1024 salidas frecuenciales al final de la etapa 10.

5. Ordenamiento de datos (Bit-reversal)

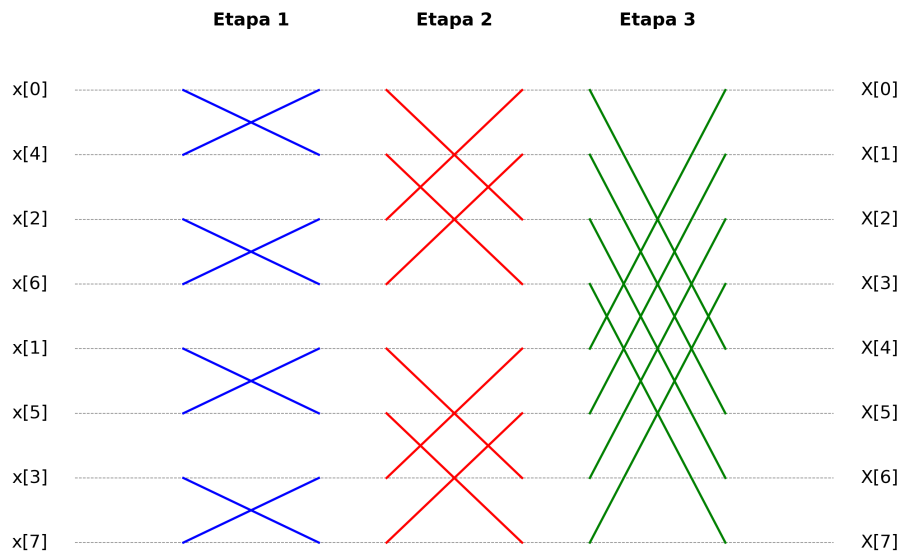
Debido a la naturaleza del esquema Radix-2 mediante decimación en tiempo (DIT), si deseamos que el vector resultante entregue el espectro de frecuencias en un orden natural progresivo ($X[0], X[1], \dots, X[1023]$), el arreglo temporal de entrada no puede ingresar a la Etapa 1 en su forma secuencial.

Es estrictamente necesario aplicar un reordenamiento inicial a los datos mediante inversión de bits (*bit-reversal*). El proceso es determinista:

1. Se toma el índice temporal de la muestra (ej. índice 1).
2. Se representa dicho índice en formato binario estricto de 10 bits para $N = 1024$ (ej. 0000000001_2).
3. Se invierte la posición de los bits como si fuera un espejo (ej. 1000000000_2).
4. Se devuelve a base decimal, reubicando la muestra original en esta nueva dirección (el índice 1 pasa a la posición 512).

6. Diagrama o esquema lógico de interconexión

Representar gráficamente las 1024 líneas y las 5,120 mariposas correspondientes generaría una saturación visual ininterpretable. Por lo tanto, se presenta el esquema general de una FFT de $N = 8$ puntos para comprender claramente la lógica subyacente. Este esquema menor (de 3 etapas) demuestra fehacientemente la conectividad de los nodos, el ordenamiento inicial invertido a las entradas, y la topología en las distancias por etapa. La proyección hacia una FFT de 1024 puntos consiste única y analógicamente en prolongar esta expansión modular de distancias y cruces hasta concluir la décima etapa.



7. Implementación del Algoritmo FFT

Para demostrar el funcionamiento integral de la arquitectura planteada, a continuación se presenta el código que emula el hardware y las iteraciones anidadas

requeridas. El siguiente bloque implementa el cálculo exacto de las 10 etapas, el reordenamiento inicial y la ejecución computacional de las 512 mariposas por etapa.

```

1 import numpy as np
2
3 def fft_1024_radix2(x):
4     N = len(x)
5     stages = int(np.log2(N)) # 10 etapas para N=1024
6
7     # 1. Ordenamiento de datos (Bit-reversal)
8     X = np.zeros(N, dtype=complex)
9     for i in range(N):
10         rev_i = int('{:010b}'.format(i)[::-1], 2)
11         X[rev_i] = x[i]
12
13     # 2. Estructura de las 10 etapas y calculo por mariposas
14     for stage in range(1, stages + 1):
15         distancia = 2 ** stage
16         half_dist = distancia // 2
17         W_N = np.exp(-2j * np.pi / distancia) # Factores de giro
18
19         # Iteracion sobre los bloques de mariposas
20         for k in range(0, N, distancia):
21             W = 1
22             for j in range(half_dist):
23                 # Operacion basica de la mariposa
24                 rama_sup = X[k + j]
25                 rama_inf = W * X[k + j + half_dist]
26
27                 # Cruces
28                 X[k + j] = rama_sup + rama_inf
29                 X[k + j + half_dist] = rama_sup - rama_inf
30
31                 # Actualizacion del factor de giro
32                 W = W * W_N
33
34     return X

```

8. Complejidad computacional

El impacto matemático de reestructurar la sumatoria mediante la lógica mostrada en el código anterior es formidable. Una DFT directa programada sin optimización para una longitud de $N = 1024$ posee una complejidad de $O(N^2)$, lo cual exige exactamente $1024^2 = 1,048,576$ multiplicaciones y sumas complejas.

Al emplear la arquitectura Radix-2 descrita, el número de operaciones se comprime a una complejidad de $O(N \log_2 N)$. Para las 10 etapas requeridas, donde se ejecutan 512 mariposas por etapa, el esfuerzo total se reduce a:

$$\text{Operaciones} = \frac{1024}{2} \log_2(1024) = 512 \times 10 = 5,120 \text{ multiplicaciones} \quad (3)$$

Este avance se traduce en realizar apenas un $\sim 0,48\%$ del costo algorítmico original. El ahorro masivo en el número de operaciones es la principal justificación técnica para el uso universal de la FFT en el procesamiento digital.

9. Conclusiones

La consolidación de este reporte evidencia de forma explícita la importancia transversal de la FFT dentro del procesamiento digital de señales. La transición algorítmica desde la evaluación discreta directa ($O(N^2)$) hacia la descomposición jerárquica mediante la operación mariposa de la FFT ($O(N \log_2 N)$) representa un ahorro que transforma problemas computacionales intratables en tareas de ejecución instantánea.

Más allá de la eficiencia matemática, el diseño ordenado en etapas concéntricas y mariposas cruzadas otorga una inmensa viabilidad práctica. Esta regularidad topológica permite no solo que sea extremadamente fácil de codificar en software mediante bucles iterativos (como Python o C), sino que su alta simetría y escalabilidad lo hacen perfecto para ser implementado físicamente a nivel de hardware, acelerando arquitecturas DSP para instrumentación médica, telecomunicaciones y análisis espectral en tiempo crítico.

Fuentes Bibliográficas

- [1] Oppenheim, A. V., & Schafer, R. W. (2010). *Tratamiento de señales en tiempo discreto* (3ra ed.). Pearson Educación.
- [2] Proakis, J. G., & Manolakis, D. G. (2007). *Tratamiento digital de señales: principios, algoritmos y aplicaciones* (4ta ed.). Pearson Prentice Hall.
- [3] Fuentes Casarrubias, O. F. (2026). *Apuntes y material del Curso de Procesamiento Digital de Señales*. Universidad Nacional Autónoma de México.